

Chapter 8. Helm and Kustomize

Kubernetes objects can be created, modified, and deleted by using imperative `kubectl` commands or by running a `kubectl` command against a manifest file declaring the desired state of an object, called a *manifest*. The primary definition language of a manifest is YAML, though you can opt for JSON, which is the less widely adopted format among the Kubernetes community. It's recommended that development teams commit and push those manifest files to version control repositories, as it will help with tracking and auditing changes over time.

Modeling an application in Kubernetes often requires a set of supporting objects, each of which can have its own manifest. For example, you may want to create a Deployment that runs the application on five Pods, a ConfigMap to inject configuration data as environment variables, and a Service for exposing network access.

It is not practical to manage a full application stack by running individual `kubectl` commands. That is where open source tools like Helm and Kustomize come into play. They allow you conveniently manage the life-cycle of application stacks and cluster components as one single unit, while at the same time allowing for contextual parameter adjustment at the time of deployment.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Use Helm and Kustomize to install cluster components
-

Working with Helm

Helm is package manager for a set of Kubernetes manifests; it also provides a templating engine. At runtime, it replaces placeholders in YAML template files with actual, end-user-defined values. The artifact produced by the Helm executable is a *chart file* that bundles the manifests that comprise the API resources of an application in the form of a TAR file. You can upload the chart file to a *chart repository* so that other teams can use it to

deploy the bundled manifests. The Helm ecosystem offers a wide range of reusable charts for common use cases searchable on [Artifact Hub](#) (for example, for running Grafana or PostgreSQL).

Due to the wealth of functionality available to Helm, we'll discuss only the basics. The exam does not expect you be a Helm expert; rather, it wants you to be familiar with the workflow of installing existing packages with Helm. Building and publishing your own charts is outside the scope of the exam. For more detailed information on Helm, see the [documentation](#). The version of Helm used to describe the functionality here is 3.19.0.

Managing an Existing Chart

As a developer, you want to reuse existing functionality instead of putting in the work to define and configure it yourself. For example, you may want to install the open source monitoring service Prometheus on your cluster.

Prometheus requires the installation of multiple Kubernetes primitives. Thankfully, the Kubernetes community provided a Helm chart, making it very easy to install and configure all the moving parts in the form of a [Kubernetes operator](#). Revisit [Chapter 7](#) to refresh your memory on the moving parts of the operator pattern.

The following list shows the typical workflow for consuming and managing a Helm chart. Most of these steps need to use the `helm` executable:

1. Identifying the chart you'd like to install
2. Adding the repository containing the chart
3. Installing the chart from the repository
4. Verifying the Kubernetes objects that have been installed by the chart
5. Rendering the list of installed charts
6. Upgrading an installed chart
7. Uninstalling a chart if its functionality is no longer needed

The following sections will explain each of the steps.

Identifying a Chart

Over the years, the Kubernetes community has implemented and published thousands of Helm charts. Artifact Hub provides a web-based search capability for discovering charts by keyword.

Say you wanted to find a chart that installs the continuous integration solution Jenkins. All you'd need to do is to enter the term *jenkins* into the

search box and press the Enter key. [Figure 8-1](#) shows the list of results in Artifact Hub.

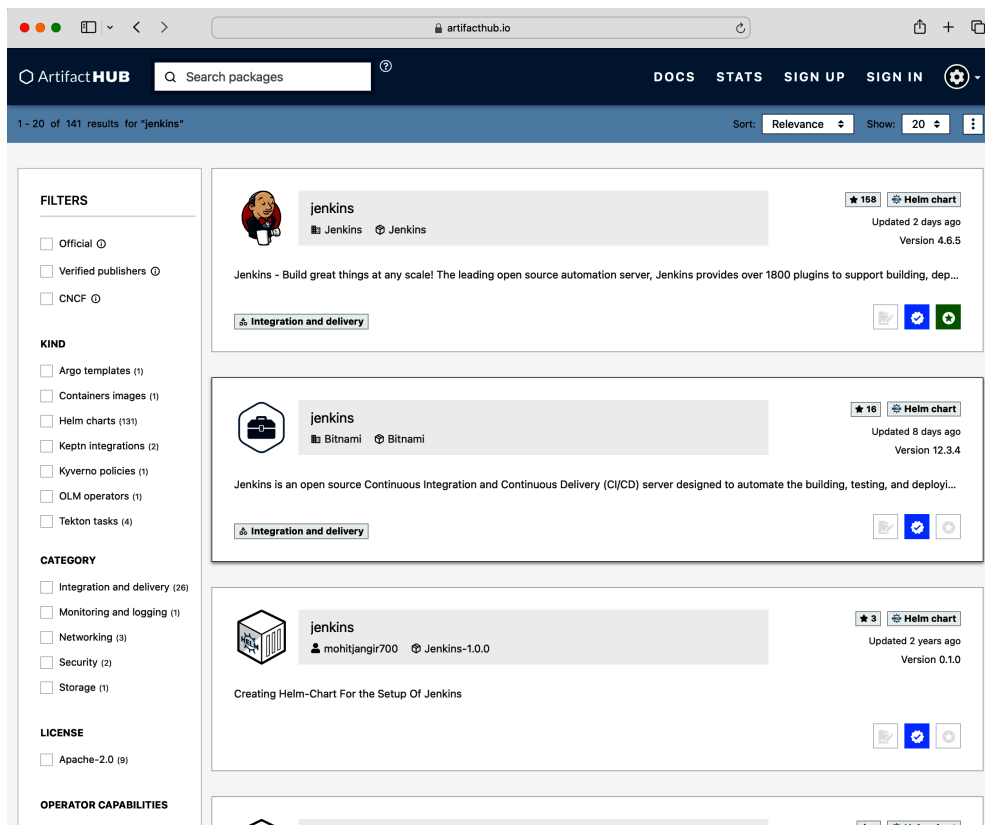


Figure 8-1. Searching for a Jenkins chart on Artifact Hub

At the time of writing, there are 141 matches for the search term. You will be able to inspect details about the chart by clicking on one of the search results, which includes a high-level description and the repository that the chart file resides in. Moreover, you can inspect the templates bundled with the chart file, indicating the objects that will be created upon installation and their configuration options. [Figure 8-2](#) shows the page for the official Jenkins chart.

You cannot install a chart directly from Artifact Hub. You must install it from the repository hosting the chart file.

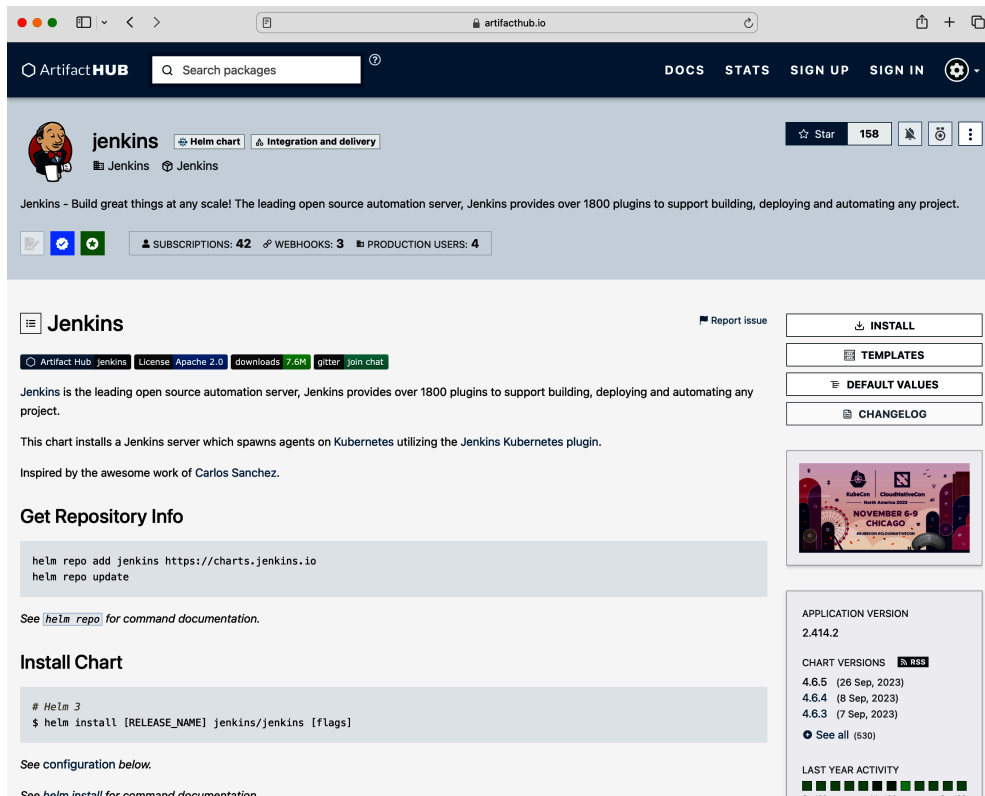


Figure 8-2. Jenkins chart details

Adding a Chart Repository

The chart description may mention the repository that hosts the chart file. Alternatively, you can click the Install button to render repository details and the command for adding it. [Figure 8-3](#) shows the contextual pop-up that appears after clicking Install.

By default, a Helm installation defines no external repositories. The following command shows how to list all registered repositories. No repositories have been registered yet:

```
$ helm repo list
```

```
Error: no repositories to show
```

As you can see from the pop-up, the chart file lives in the repository with the URL <https://charts.jenkins.io>. We will need to add this repository. This is a one-time operation. You can install other charts from that repository or you can update a chart that originated from that repository with commands discussed in a later section.



Figure 8-3. Jenkins chart installation instructions

You need to provide a name for the repository when registering one. Make the repository name as descriptive as possible. The following command registers the repository with the name `jenkinsci`:

```
$ helm repo add jenkinsci https://charts.jenkins.io/
"jenkinsci" has been added to your repositories
```

Listing the repositories now shows the mapping between name and URL:

```
$ helm repo list
NAME          URL
jenkinsci     https://charts.jenkins.io/
```

You permanently added the repository to the Helm installation.

Searching for a Chart in a Repository

The Install pop-up window already provided the command to install the chart. You can also search the repository for available charts in case you do not know their names or latest versions. Add the `--versions` flag to list all available versions:

```
$ helm search repo jenkinsci
NAME          CHART VERSION  APP VERSION  DESCRIPTION
jenkinsci/jenkins  5.8.26        2.492.2      ...
```

At the time of writing, the latest version available is 5.8.26. This may be different if you run the command on your machine, given that the Jenkins project may have released a newer version.

Installing a Chart

Let's assume that the latest version of the Helm chart contains a security vulnerability. Therefore, we decide to install the Jenkins chart with the previous version, 5.8.25. You need to assign a name to be able to identify an installed chart. The name we'll use here is `my-jenkins`:

```
$ helm install my-jenkins jenkinsci/jenkins --version 5.8.25
NAME: my-jenkins
LAST DEPLOYED: Wed Mar 26 13:48:50 2025
NAMESPACE: default
STATUS: deployed
REVISION: 1
...
```

The chart automatically created the Kubernetes objects in the `default` namespace. You can use the following command to discover the most important resource types:

```
$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/my-jenkins-0	2/2	Running	0	12m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	..
service/my-jenkins	ClusterIP	10.99.166.189	<none>	..
service/my-jenkins-agent	ClusterIP	10.110.246.141	<none>	..

NAME	READY	AGE
statefulset.apps/my-jenkins	1/1	12m

The chart has been installed with the default configuration options. You can inspect those default values by clicking the Default Values button on the chart page, as shown in [Figure 8-4](#).

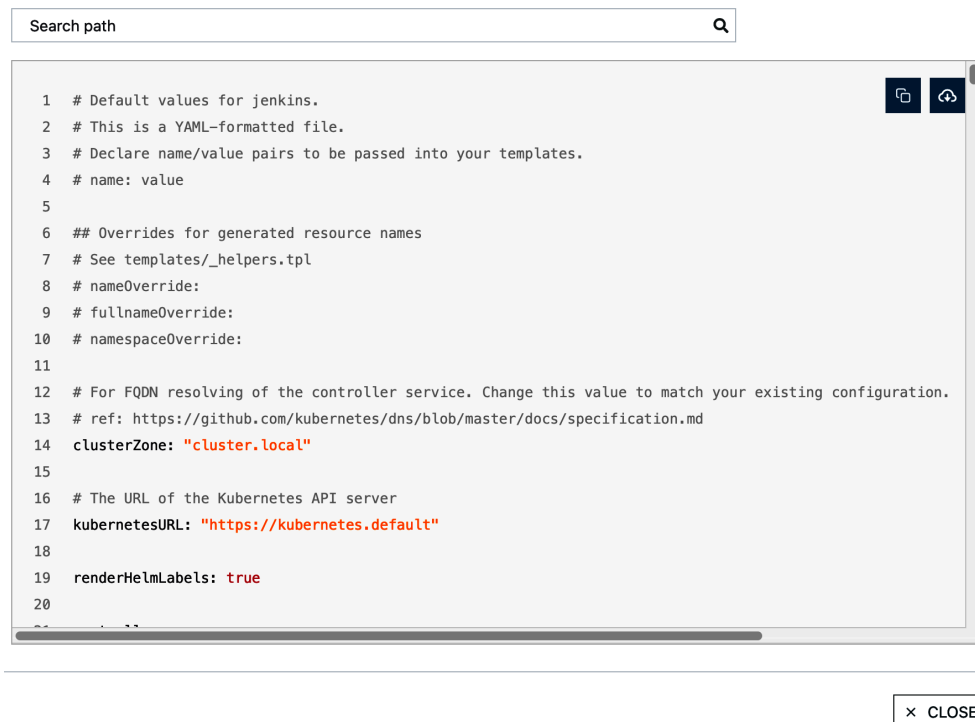


Figure 8-4. Jenkins chart default values

You can also discover those configuration options using the following command. The output shown renders only a subset of values, the admin username and its password, represented by `controller.adminUser` and `controller.adminPassword`:

```
$ helm show values jenkinsci/jenkins
```

```
...
```

```
controller:
```

```

# When enabling LDAP or another non-Jenkins identity source, the built-
# admin account will no longer exist.
# If you disable the non-Jenkins identity store and instead use the Jenl
# internal one,
# you should revert controller.adminUser to your preferred admin user:
adminUser: "admin"
# adminPassword: <defaults to random>

```

```
...
```

You can customize any configuration value when installing the chart. To pass configuration data during the install processing, use one of the following flags:

```
--values
```

Specifies the overrides in the form of a pointer to a YAML manifest file

```
--set
```

Specifies the overrides directly from the command line

For more information, see [“Customizing the Chart Before Installing”](#) in the Helm documentation.

You can decide to install the chart into a custom namespace. Use the `-n` flag to provide the name of an existing namespace. Add the flag `--create-namespace` to automatically create the namespace if it doesn't exist yet.

The following command shows how to customize some of the values and the namespace used during the installation process:

```
$ helm install my-jenkins jenkinsci/jenkins --version 4.6.4 \
  --set controller.adminUser=boss --set controller.adminPassword=password \
  -n jenkins --create-namespace
```

We specifically set the username and the password for the admin user. Helm created the objects controlled by the chart into the `jenkins` namespace.

Listing Installed Charts

Charts can live in the `default` namespace or a custom namespace. You can inspect the list of installed charts using the `helm list` command. If you do not know which namespace, simply add the `--all-namespaces` flag to the command:

```
$ helm list --all-namespaces
```

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART
my-jenkins	default	1	2023-09-28...	deployed	jenkins-4.6.0

The output of the command includes the column `NAMESPACE` that shows the namespace used by a particular chart. Similar to the use of `kubectl`, the `helm list` command provides the option `-n` for spelling out a namespace. Providing no flag(s) with the command will return the result for the `default` namespace.

Upgrading an Installed Chart

Upgrading an installed chart usually means moving to a new chart version. You can poll for new versions available in the repository by running this command:

\$ helm repo update

```
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "jenkinsci" chart repository
Update Complete. *Happy Helming!*
```

What if you want to upgrade your existing chart installation to a newer chart version? Run the following command to upgrade the chart to that specific version with the default configuration:

```
$ helm upgrade my-jenkins jenkinsci/jenkins --version 5.8.26
Release "my-jenkins" has been upgraded. Happy Helming!
...
```

As with the `install` command, you will have to provide custom configuration values if you want to tweak the chart's runtime behavior when upgrading a chart.

Uninstalling a Chart

Sometimes you no longer need to run a chart. The command for uninstalling a chart is straightforward, as shown here. It will delete all objects controlled by the chart. Don't forget to provide the `-n` flag if you previously installed the chart into a namespace other than `default`:

```
$ helm uninstall my-jenkins
release "my-jenkins" uninstalled
```

Executing the command may take up to 30 seconds, as Kubernetes needs to wait for the workload grace period to end.

Working with Kustomize

Kustomize is a tool introduced with Kubernetes 1.14 that aims to make manifest management more convenient. It supports three different use cases:

- Generating manifests from other sources. For example, creating a ConfigMap and populating its key-value pairs from a properties file.
- Adding common configuration across multiple manifests. For example, adding a namespace and a set of labels for a Deployment and a Service.

- Composing and customizing a collection of manifests. For example, setting resource boundaries for multiple Deployments.

The central file needed for Kustomize to work is the kustomization file. The standardized name for the file is *kustomization.yaml* and cannot be changed. A kustomization file defines the processing rules Kustomize works upon.

Kustomize is fully integrated with `kubectl` and can be executed in two modes: rendering the processing output on the console or creating the objects. Both modes can operate on a directory, tarball, Git archive, or URL as long as they contain the kustomization file and referenced resource files:

Rendering the produced output

The first mode uses the `kustomize` subcommand to render the produced result on the console but does not create the objects. This command works similarly to the dry-run option you might know from the `run` command:

```
$ kubectl kustomize <target>
```

Creating the objects

The second mode uses the `apply` command in conjunction with the `-k` command-line option to apply the resources processed by Kustomize, as explained in the previous section:

```
$ kubectl apply -k <target>
```

The following sections demonstrate each of the use cases by a single example. For a full coverage on all possible scenarios, refer to the [documentation](#) or the [Kustomize GitHub repository](#).

Composing Manifests

One of the core functionalities of Kustomize is to create a composed manifest from other manifests. Combining multiple manifests into a single one may not seem that useful by itself, but many of the other features described later will build upon this capability. Say you wanted to compose a single YAML definition with multiple manifests separated by “---” from a Deployment and a Service resource file. All you need to do is to place the resource files into the same folder as the kustomization file:

```

.
├─ kustomization.yaml
├─ web-app-deployment.yaml
└─ web-app-service.yaml

```

The kustomization file lists the resources in the `resources` section, as shown in [Example 8-1](#).

Example 8-1. A kustomization file combining two manifests

```

resources:
- web-app-deployment.yaml
- web-app-service.yaml

```

As a result, the `kustomize` subcommand renders the combined manifest containing all of the resources separated by three hyphens (`---`) to denote the different object definitions:

```

$ kubectl kustomize ./
apiVersion: v1
kind: Service
metadata:
  labels:
    app: web-app-service
  name: web-app-service
spec:
  ports:
  - name: web-app-port
    port: 3000
    protocol: TCP
    targetPort: 3000
  selector:
    app: web-app
  type: NodePort
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: web-app-deployment
  name: web-app-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:

```

```

labels:
  app: web-app
spec:
  containers:
  - env:
    - name: DB_HOST
      value: mysql-service
    - name: DB_USER
      value: root
    - name: DB_PASSWORD
      value: password
    image: bmuschko/web-app:1.0.1
    name: web-app
  ports:
  - containerPort: 3000

```

Generating Manifests from Other Sources

Earlier in this chapter, we learned that ConfigMaps and Secrets can be created by pointing them to a file containing the actual configuration data for it. Kustomize can help with the process by mapping the relationship between the YAML manifest of those configuration objects and their data. Furthermore, we'll want to inject the created ConfigMap and Secret in a Pod as environment variables. In this section, you will learn how to achieve this with the help of Kustomize.

The following file and directory structure contains the YAML file for the Pod and the configuration data files we need for the ConfigMap and Secret. The mandatory kustomization file lives on the root level of the directory tree:

```

.
├── config
│   ├── db-config.properties
│   └── db-secret.properties
├── kustomization.yaml
└── web-app-pod.yaml

```

In *kustomization.yaml*, you can define that the ConfigMap and Secret object should be generated with the given name. The name of the ConfigMap is supposed to be `db-config`, and the name of the Secret is going to be `db-creds`. Both of the generator attributes, `configMapGenerator` and `secretGenerator`, reference an input file used to feed in the configuration data. Any additional resources can be spelled out with the `resources` attribute. [Example 8-2](#) shows the contents of the kustomization file.

Example 8-2. A kustomization file using a ConfigMap and Secret generator

```
configMapGenerator:
- name: db-config
  files:
  - config/db-config.properties
secretGenerator:
- name: db-creds
  files:
  - config/db-secret.properties
resources:
- web-app-pod.yaml
```

Kustomize generates ConfigMaps and Secrets by appending a suffix to the name. You can see this behavior when creating the objects using the `apply` command. The ConfigMap and Secret can be referenced by name in the Pod manifest:

```
$ kubectl apply -k ./
configmap/db-config-t4c79h4mtt unchanged
secret/db-creds-4t9dmgtf9h unchanged
pod/web-app created
```

CONFIGURING THE NAMING STRATEGY

This naming strategy can be configured with the attribute `generatorOptions` in the kustomization file. See the [documentation](#) for more information.

Let's also try the `kustomize` subcommand. Instead of creating the objects, the command renders the processed output on the console:

```
$ kubectl kustomize ./
apiVersion: v1
data:
  db-config.properties: |-
    DB_HOST: mysql-service
    DB_USER: root
kind: ConfigMap
metadata:
  name: db-config-t4c79h4mtt
---
apiVersion: v1
data:
  db-secret.properties: REJfUEFTU1dPUkQ6IGNHRnpjM2R2Y2lRPQ==
kind: Secret
metadata:
```

```

    name: db-creds-4t9dmgtf9h
type: Opaque
---
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: web-app
    name: web-app
spec:
  containers:
  - envFrom:
    - configMapRef:
        name: db-config-t4c79h4mtt
    - secretRef:
        name: db-creds-4t9dmgtf9h
    image: bmuschko/web-app:1.0.1
    name: web-app
    ports:
    - containerPort: 3000
      protocol: TCP
  restartPolicy: Always

```

Adding Common Configuration Across Multiple Manifests

Application developers usually work on an application stack set comprised of multiple manifests. For example, an application stack could consist of a frontend microservice, a backend microservice, and a database. It's common practice to use the same cross-cutting configuration for each of the manifests. Kustomize offers a range of supported fields (e.g., namespace, labels, or annotations). Refer to the [documentation](#) to learn about all supported fields.

For the next example, we'll assume that a Deployment and a Service live in the same namespace and use a common set of labels. The namespace is called `persistence` and the label is the key-value pair `team: helix`. [Example 8-3](#) illustrates how to set those common fields in the kustomization file.

Example 8-3. A kustomization file using a common field

```

namespace: persistence
commonLabels:
  team: helix
resources:
- web-app-deployment.yaml
- web-app-service.yaml

```

To create the referenced objects in the kustomization file, run the `apply` command. Make sure to create the `persistence` namespace beforehand:

```
$ kubectl create namespace persistence
namespace/persistence created
$ kubectl apply -k ./
service/web-app-service created
deployment.apps/web-app-deployment created
```

The YAML representation of the processed files looks as follows:

```
$ kubectl kustomize ./
apiVersion: v1
kind: Service
metadata:
  labels:
    app: web-app-service
    team: helix
  name: web-app-service
  namespace: persistence
spec:
  ports:
  - name: web-app-port
    port: 3000
    protocol: TCP
    targetPort: 3000
  selector:
    app: web-app
    team: helix
  type: NodePort
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: web-app-deployment
    team: helix
  name: web-app-deployment
  namespace: persistence
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
      team: helix
  template:
    metadata:
      labels:
        app: web-app
```

```

    team: helix
spec:
  containers:
  - env:
    - name: DB_HOST
      value: mysql-service
    - name: DB_USER
      value: root
    - name: DB_PASSWORD
      value: password
    image: bmuschko/web-app:1.0.1
    name: web-app
  ports:
  - containerPort: 3000

```

Customizing a Collection of Manifests

Kustomize can merge the contents of a YAML manifest with a code snippet from another YAML manifest. Typical use cases include adding security context configuration to a Pod definition or setting resource boundaries for a Deployment. The kustomization file allows for specifying different patch strategies like `patchesStrategicMerge` and `patchesJson6902`. For a deeper discussion on the differences between patch strategies, refer to the [Kubernetes documentation](#).

[Example 8-4](#) shows the contents of a kustomization file that patches a Deployment definition in the file `nginx-deployment.yaml` with the contents of the file `security-context.yaml`.

Example 8-4. A kustomization file defining a patch

```

resources:
- nginx-deployment.yaml
patchesStrategicMerge:
- security-context.yaml

```

The patch file shown in [Example 8-5](#) defines a security context on the container-level for the Pod template of the Deployment. At runtime, the patch strategy tries to find the container named `nginx` and enhances the additional configuration.

Example 8-5. The patch YAML manifest

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment

```



```
spec:
  template:
    spec:
      containers:
      - name: nginx
        securityContext:
          runAsUser: 1000
          runAsGroup: 3000
          fsGroup: 2000
```

The result is a patched Deployment definition, as shown in the output of the `kustomize` subcommand shown next. The patch mechanism can be applied to other files that require a uniform security context definition:

```
$ kubectl kustomize ./
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: nginx
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.14.2
        name: nginx
        ports:
        - containerPort: 80
        securityContext:
          fsGroup: 2000
          runAsGroup: 3000
          runAsUser: 1000
```

Key Differences Between Helm and Kustomize

On the surface, Helm and Kustomize may seem to solve the same problems. Helm is about packaging and distribution, Kustomize is about configuration management. Use both where each fits best.

Here, we want to identify the key difference between those tools so that you can make an educated decision based on the use cases you are trying to fulfill:

Ease of use

Kustomize is bundled with the `kubectl` command line. You do not need to install another tool, nor do you have learn another templating engine.

Helm requires the installation of an executable and requires you to become familiar with its commands and workflow.

Learning curve

Kustomize builds upon the knowledge of an administrator or developer familiar with writing Kubernetes YAML manifests.

Helm has a steeper learning curve. You will have to become familiar with its package management system, the notation for its templating engine, and how to pass in user-defined values upon invocation.

Packaging

Kustomize doesn't require the end user to produce an archive file. All you need is a set of YAML manifests and the *kustomization.yaml* file, which you would check into a Git repository.

Helm, on the other hand, requires the creation of a metadata file named *Chart.yaml*, default values represented in a file named *values.yaml*, and a set of template files in a *templates* subdirectory. To be able to distribute the Helm chart, you'll need to package it into a TAR file.

Release versioning

Kustomize solely focuses on generating the desired state in a cluster through YAML manifests. You can track changes over time by Git commit hashes or tags to indicate a version.

Helm's rigid project structure requires the definition of a chart version inside of the *Chart.yaml* file. Every time you make a change, you'd bump up the version number, often represented by semantic versioning.

Helm and Kustomize are Kubernetes tools used to automate the process of deploying objects into a cluster. Carefully analyze your technical requirements, business objectives, and the skillset of team members before

deciding on either of the tools. You may even determine that you should use both tools to manage application stacks in Kubernetes. Most Kubernetes GitOps tools, e.g., Argo CD or Flux, support Helm and Kustomize.

Summary

Helm has evolved to become a de facto tool for deploying application stacks to Kubernetes. The artifact that contains the manifest files, default configuration values, and metadata is called a chart. A team or an individual can publish charts to a chart repository. Users can discover a published chart through the Artifact Hub user interface and install it to a Kubernetes cluster.

One of the primary developer workflows when using Helm consists of finding, installing, and upgrading a chart with a specific version. You start by registering the repository containing chart files you want to consume. The `helm install` command downloads the chart file and stores it in a local cache. It also creates the Kubernetes objects described by the chart.

The installation process is configurable. A developer can provide overrides for customizable configuration values. The `helm upgrade` command lets you upgrade the version of an already installed chart. To uninstall a chart and delete all Kubernetes objects managed by the chart, run the `helm uninstall` command.

Additional tools emerged for more convenient manifest management. Kustomize is fully integrated with the `kubectl` tool chain. It helps with the generation, composition, and customization of manifests.

Exam Essentials

Assume that the Helm and Kustomize executables are preinstalled

Unfortunately, the [exam FAQ](#) does not mention any details about the Helm and Kustomize executables. It's fair to assume that they will be preinstalled for you, and therefore you do not need to memorize installation instructions.

Become familiar with Artifact Hub

Artifact Hub provides a web-based UI for Helm charts. It's worthwhile to explore the search capabilities and the details provided by individual charts, more specifically the repository the chart file lives in, and its configurable values. During the exam, you'll likely

not be asked to navigate to Artifact Hub because its URL hasn't been listed as one of the permitted documentation pages. You can assume that the exam question will provide you with the repository URL.

Practice commands needed to consume existing Helm charts

The exam does not ask you to build and publish your own chart file. All you need to understand is how to consume an existing chart. You will need to be familiar with the `helm repo add` command to register a repository, the `helm search repo` to find available chart versions, and the `helm install` command to install a chart. You should have a basic understanding of the upgrade process for an already installed Helm chart using the `helm upgrade` command.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. In this exercise, you will use Helm to install Kubernetes objects needed for the open source monitoring solution [Prometheus](#). The easiest way to install Prometheus on top of Kubernetes is with the help of the [prometheus-operator](#) Helm chart.

You can search for the [kube-prometheus-stack](#) on Artifact Hub. Add the repository to the list of known repositories accessible by Helm with the name `prometheus-community`.

Update to the latest information about charts from the respective chart repository.

Run the Helm command for listing available Helm charts and their versions. Identify the latest chart version for `kube-prometheus-stack`.

Install the chart `kube-prometheus-stack`. List the installed Helm chart.

List the Service named `prometheus-operated` created by the Helm chart. The object resides in the `default` namespace.

Use the `kubectl port-forward` command to forward the local port 8080 to the port 9090 of the Service. Open a browser and bring up the Prometheus dashboard.

Stop port forwarding and uninstall the Helm chart.

2. Create the directory named *manifests*. Within the directory, create two files: *pod.yaml* and *configmap.yaml*. The *pod.yaml* file should define a Pod named `nginx` with the image `nginx:1.21.1`. The *configmap.yaml* file defines a ConfigMap named `logs-config` with the

key-value pair `dir=/etc/logs/traffic.log`. Create both objects with a single, declarative command.

Modify the ConfigMap manifest by changing the value of the key `dir` to `/etc/logs/traffic-log.txt`. Apply the changes. Delete both objects with a single declarative command.

Use Kustomize to set a common namespace `test` for the resource file *pod.yaml*. The file *pod.yaml* defines the Pod named `nginx` with the image `nginx:1.21.1` without a namespace. Run the Kustomize command that renders the transformed manifest on the console.